

In Excel VBA, **Events** are specific actions performed by a user or by Excel itself—such as opening a workbook, clicking a cell, or changing a value—that trigger a macro to run automatically. Instead of manually clicking a button to start your code, events allow your spreadsheets to become truly interactive and "intelligent."

1. Where to Write Event Code

Unlike standard macros stored in "Modules," event code must be placed in specific object modules within the **Visual Basic Editor (Alt + F11)**:

- **ThisWorkbook**: For actions affecting the entire file (e.g., Opening, Closing, Saving).
 - **Sheet Modules (e.g., Sheet1)**: For actions affecting a specific worksheet (e.g., Selecting a cell, Changing a value).
-

2. Common Worksheet Events

These events live inside the code window for a specific sheet. They are essential for tasks like real-time **Data Validation** or building dynamic dashboards.

- **Worksheet_SelectionChange**: Triggers every time you move the active cell. You can use this to highlight the current row or display a **Message Box** with context-sensitive help.
 - **Worksheet_Change**: Triggers whenever the *content* of a cell is modified. This is the gold standard for creating audit trails or automatically updating a **Sales chart** when new data is entered.
 - **Worksheet_Activate/Deactivate**: Runs code when you switch to or away from a specific tab, such as refreshing a **Pivot Table** every time you view the "Summary" sheet.
-

3. Common Workbook Events

These events apply to the file as a whole and are found in the **ThisWorkbook** module.

- **Workbook_Open**: Automatically runs when the file is launched. Use this to display a "Welcome" message, set specific **Window hacks** like Zoom levels, or check if the user has the correct permissions.
 - **Workbook_BeforeClose**: Triggers just before the user exits. It is perfect for ensuring the user has filled in all mandatory fields in a "Salary statement" or for automatically creating a backup.
 - **Workbook_BeforePrint**: Allows you to verify **Printing options** or update headers and footers right before the document hits the printer.
-

4. Preventing Infinite Loops

A common trap with events is the **Infinite Loop**. For example, if a `Worksheet_Change` macro changes a cell value, it triggers itself again, potentially crashing Excel.

The Solution:

Use `Application.EnableEvents = False` at the start of your code to "silence" Excel while your macro is working, then set it back to `True` at the end.

VBA

```
Private Sub Worksheet_Change(ByVal Target As Range)
    On Error Resume Next
    Application.EnableEvents = False

    ' Your code here (e.g., making text uppercase)
    Target.Value = UCase(Target.Value)

    Application.EnableEvents = True
End Sub
```

5. Practical Engineering Use Cases

- **Auto-Timestamping:** Use `Worksheet_Change` to automatically put the current date and time in Column B whenever a "Student Name" is entered in Column A.
- **Form Security:** Use `Workbook_Open` to hide sensitive worksheets (like a master salary list) and only show them after a user enters a password via an **Input Box**.
- **Dynamic Navigation:** Use `Worksheet_SelectionChange` so that clicking a cell with a project name automatically takes the user to that project's detailed sheet via a **Hyperlink** action.